Complex Algorithm Simulator
Analysis and Validation Report

Jack Brunswik

April 5, 2026

1 Introduction

This simulation project builds on the stochastic algorithm simulation framework developed in Milestone 3, which supports Randomized Quick Sort, Random Graph BFS, and Karger's Min-Cut. The system provides Monte Carlo simulation, visualization, and histogram-based analysis to study algorithm behavior under varying input sizes and randomized conditions.

The goal of this milestone is to analyze how different parameters affect algorithm performance, evaluate system behavior under multiple scenarios, and validate that the simulation produces reasonable and expected results. The analysis focuses on key metrics such as comparison counts for Quick Sort, tree depth for BFS, and success probability for Karger's Min-Cut.

Using the data collected in Milestone 3, this report explores how input size (n) and number of trials (k) influence performance, variability, and reliability across the algorithms. The results are used to assess the sensitivity of the system, identify meaningful patterns, and evaluate the accuracy of the simulation relative to theoretical expectations.

2 Parameter Analysis

The primary parameters analyzed in this simulation are input size (n) and number of trials (k). These parameters directly influence both the performance and variability of the algorithms.

Input size (n) has the strongest impact on output behavior. For Randomized Quick Sort, increasing n results in a rapid increase in the number of comparisons, closely following the expected O(n log n) growth pattern observed in Milestone 3. For example, as n increases from 500 to 1000, the mean number of comparisons increases significantly from approximately 4600 to 10800, demonstrating high sensitivity to input size (graphs seen below).

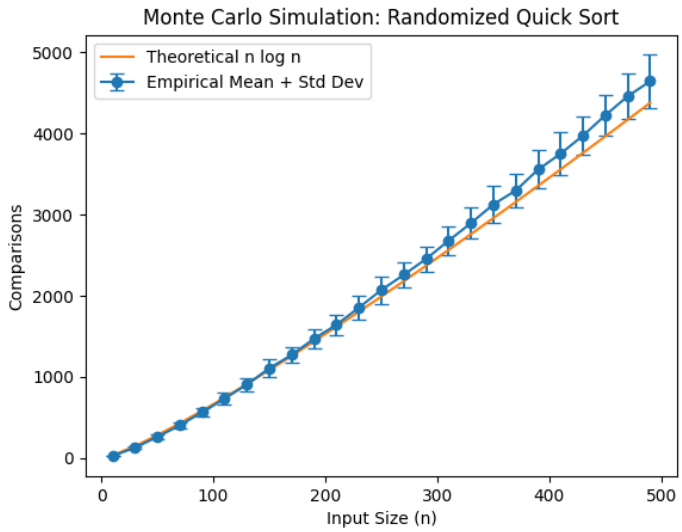

Figure 1: Quick Sort simulated in Monte Carlo mode with max input set to 500

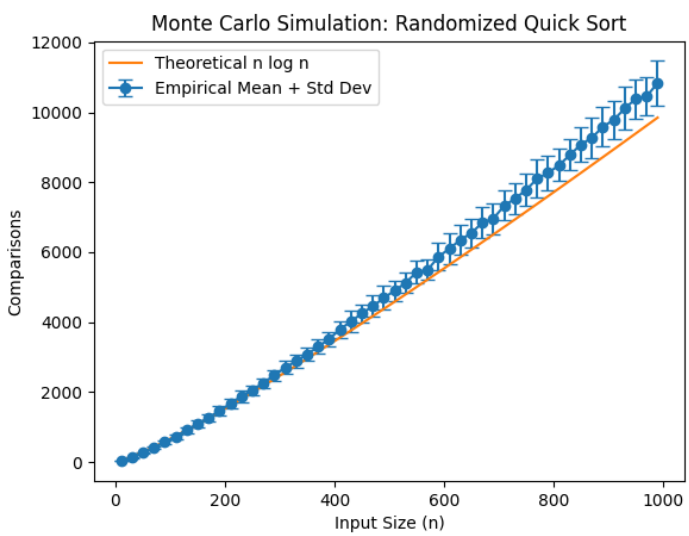

Figure 2: Quick Sort simulated in Monte Carlo mode with max input set to 1000

In contrast, Random Graph BFS shows much lower sensitivity to input size. As n increases, the average tree depth grows slowly, consistent with logarithmic behavior. For instance, increasing n from 500 to 1000 results in only a small increase in mean tree depth (from approximately 12.44 to 14.11), indicating low sensitivity (graphs seen below).

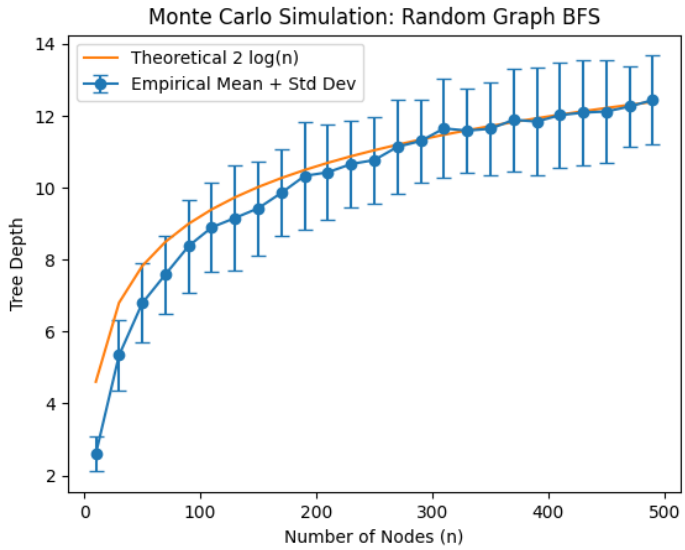

Figure 3: Graph BFS simulated in Monte Carlo mode with max input set to 500

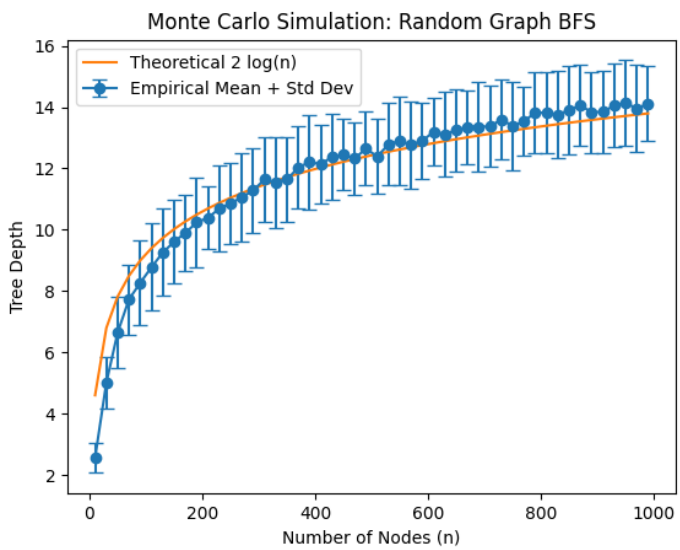

Figure 4: Graph BFS simulated in Monte Carlo mode with max input set to 1000

For Karger's Min-Cut algorithm, a modified Monte Carlo approach was used to obtain stable probability estimates. Initially, using a single fixed graph per input size resulted in extreme outcomes, where the success probability was often

either 0.0 or 1.0 due to the algorithm's strong dependence on graph structure. To address this, multiple random graphs were generated for each input size, and for each graph, the true minimum cut was computed once using a deterministic method. The algorithm was then executed multiple times on that same graph to estimate the probability of successfully identifying the minimum cut. These probabilities were averaged across all generated graphs to produce the final empirical estimate. This approach reduces bias from any single graph instance, avoids extreme probability values, and more accurately reflects the expected behavior of the algorithm across a distribution of inputs.

As for the parameter effects, the number of trials (k) primarily affects the stability and reliability of results rather than the mean itself. Increasing k reduces variability and leads to more consistent estimates. This is especially important for Karger's Min-Cut, where the success probability stabilizes as k increases. As shown in the below results, increasing trials from 20 to 200 improves the consistency of success probability estimates, demonstrating convergence behavior.

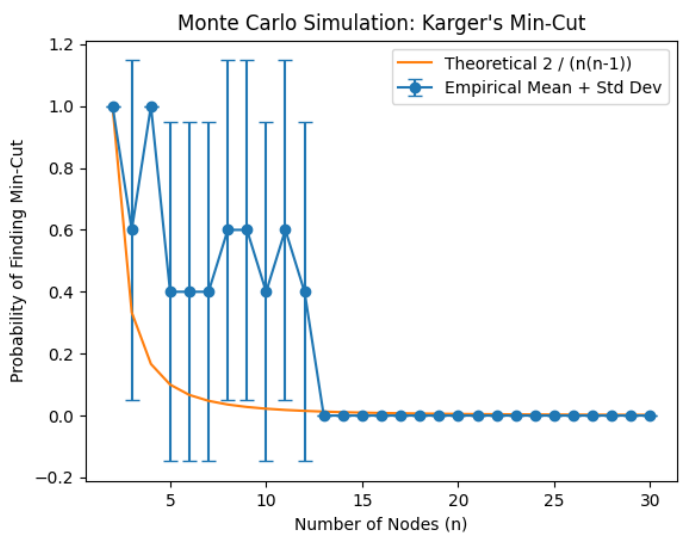

Figure 5: Karger simulated in Monte Carlo mode with trials input set to 20

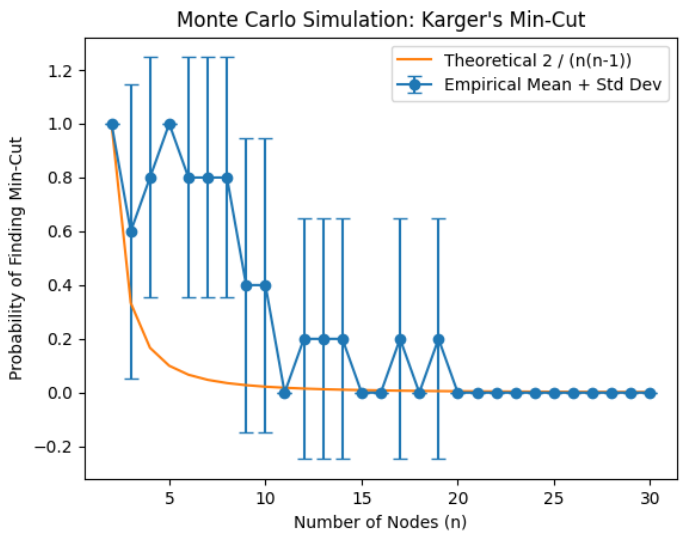

Figure 6: Karger simulated in Monte Carlo mode with trials input set to 200

The step parameter controls the increment between input sizes (n) used in the simulation and determines the number of data points collected. A smaller step size results in more frequent sampling of input values, producing smoother and more detailed graphs that better capture gradual changes in algorithm behavior. However, this increases computational cost, as the simulation must be executed more times. Conversely, a larger step size reduces the number of simulations required, improving performance but producing less detailed results. The changes in steps can be seen within the Quick Sort and Graph BFS figures above.

Overall, input size (n) is the most influential parameter for performance growth, while the number of trials (k) primarily affects statistical reliability.

3 Scenario Testing

To evaluate system behavior under different conditions, three scenarios were tested: normal conditions, high-load conditions, and high-trial conditions.

Note: All data obtained for the following scenarios are recorded and stored in the "M4 Data Files" provided with this report.

3.1 Scenario 1: Normal Operating Conditions

In this scenario, the system was evaluated under typical input conditions to establish baseline performance across all algorithms. The simulation was run with input sizes ranging from n = 30 or n = 200, a step size of 1 or 10, and k = 50 trials.

Results:

- Randomized Quick Sort comparisons increased steadily with input size, following expected O(n log n) behavior.
- Random Graph BFS tree depth showed minimal growth, remaining relatively small even as n increased.
- Karger's Min-Cut success probability decreased as n increased, with moderate probabilities at smaller input sizes and lower values at larger sizes.
- All algorithms produced stable and consistent results with low variability.

3.2 Scenario 2: High Input Size (Stress Test)

This scenario examined system performance under larger input sizes to evaluate scalability and computational limits. The simulation was run with separate input sizes for Quick Sort and BFS, and Karger's. Inputs ranged from max n = 5000 to n = 10000, a step size of 100, and k = 300 trials (Quick Sort and BFS), and max n = 200, a step size of 10, and k = 50 trials (Karger's).

Results:

- Randomized Quick Sort showed a significant increase in comparisons as n increased, reflecting its theoretical complexity.
- Random Graph BFS maintained efficient performance, with only slight increases in tree depth even at large input sizes.
- Karger's Min-Cut success probability dropped sharply, often reaching very low values at higher n.
- Computational cost increased noticeably for Karger's algorithm compared to the others.

3.3 Scenario 3: High Trial Count (Statistical Stability)

This scenario focused on how the number of trials (k) affects the stability and reliability of results. Simulations were run with fixed input sizes (max n = 30 or n = 200, step is 1 or 10) while varying k across multiple runs (k = 50, 200, 500).

Results:

- Randomized Quick Sort mean comparisons remained consistent across different values of k.
- Random Graph BFS mean tree depth also remained stable regardless of k.

- Variability in results decreased as k increased, producing smoother and more consistent curves.
- Karger's Min-Cut maintained the same decreasing trend with respect to n, but higher k values reduced fluctuations in probability estimates.

4 Validation

To ensure the correctness and reliability of the simulation model, multiple validation methods were applied. These methods evaluate how closely the simulation aligns with theoretical expectations, how consistent the results are, and identify any limitations in the modeling approach.

4.1 Theoretical Comparison

Simulation outputs were compared against known theoretical expectations for each algorithm. Randomized Quick Sort exhibited growth consistent with O(n log n), and Random Graph BFS showed logarithmic growth in tree depth. Karger's Min-Cut success probability followed the expected decreasing trend proportional to 2/(n(n - 1)). While minor deviations were observed due to randomness and finite sampling, the overall trends closely matched theoretical predictions. This alignment provides strong evidence that the model produces reasonable and accurate results.

A limitation of this method is that theoretical curves represent asymptotic behavior, while the simulation operates on finite input sizes. As a result, small discrepancies are expected, particularly at lower values of n.

4.2 Consistency Across Trials

The simulation was executed multiple times with identical parameters to verify consistency. The mean outputs for all algorithms remained stable across runs, while variability decreased as the number of trials (k) increased. This demonstrates convergence of the Monte Carlo estimates and supports the reliability of the results.

However, for smaller values of k, noticeable fluctuations were observed, particularly for Karger's Min-Cut due to its low success probability. This highlights a limitation of Monte Carlo methods, where insufficient sampling can lead to unstable or less reliable estimates.

4.3 Sensitivity Analysis Validation

Parameter sensitivity was evaluated by varying input size (n) and the number of trials (k). The results showed that increasing n significantly impacted Quick Sort and Karger's Min-Cut, while BFS remained relatively insensitive. Increasing k

improved stability without significantly affecting mean values. These responses are consistent with theoretical expectations, confirming that the model behaves appropriately under parameter changes.

A limitation of this analysis is that only a subset of parameters was varied independently. Other factors, such as graph structure in Karger's algorithm, can also influence results but were not exhaustively explored.

Overall, the strong agreement between empirical results and theoretical expectations, combined with consistent behavior across repeated trials and reasonable sensitivity to parameter changes, provides convincing evidence that the simulation model is valid. The identified limitations are inherent to stochastic simulation and do not significantly undermine the overall reliability of the results.

5 Statistical Summary

Statistical analysis was conducted to evaluate the consistency and variability of the simulation results. For each algorithm, key metrics such as mean, standard deviation, and range were examined.

Randomized Quick Sort showed moderate variability, with standard deviation increasing as input size increased. For example, at max n = 500, the mean number of comparisons was approximately 4800 with a high standard deviation. This indicates that while results vary due to randomness, they remain relatively consistent.

Random Graph BFS exhibited very low variability, with a low standard deviation present even for large input sizes. This suggests that BFS behavior is highly predictable across different random graph instances.

Karger's Min-Cut showed the highest variability, with standard deviation levels reflecting the binary nature of success or failure. For example, at n = 60, the mean success probability was low, and variability remained high due to the stochastic nature of the algorithm.

Overall, the statistical results indicate that Quick Sort and BFS produce stable and reliable outputs, while Karger's Min-Cut requires a larger number of trials to achieve consistent results.

6 Conclusion

This analysis demonstrates that the simulation framework effectively models the behavior of stochastic algorithms and produces results consistent with theoretical expectations. Input size (n) was identified as the most influential parameter, significantly affecting performance across all algorithms, while the number of trials (k) primarily influenced result stability and convergence.

Randomized Quick Sort and Random Graph BFS showed predictable and efficient behavior across all tested conditions, with low to moderate variability. In contrast, Karger's Min-Cut exhibited high variability and computational cost, making it less practical for large input sizes without optimization.

The model performed well under both normal and extreme conditions, validating its correctness and usefulness as an analysis tool. However, limitations were observed in scalability, particularly for computationally expensive algorithms like Karger's Min-Cut.

Future improvements could include optimizing algorithm implementations and introducing parallel processing. Overall, the simulation provides a sturdy platform for analyzing stochastic algorithm behavior and understanding the impact of randomness on performance.